xyndrome LMS Beginner Owner Guide

A plain-English manual for opening, understanding, changing, and launching your app

Read this first. You do not need to become a software engineer to own this project. This guide explains the project like a map: where it lives, what each major folder means, how to open it locally, what Git main and branches mean, and what still matters before launch.

Item	Meaning
Project folder	/Applications/XAMPP/xamppfiles/htdocs/lms
Local website	http://localhost/lms/
Login page	http://localhost/lms/auth/login
Backend API	http://localhost:3000/api
Current Git branch	codex/backup-global-card-style-20260530-191847
GitHub remote	https://github.com/laheesahamed-cloud/lms.git
Generated on	2026-05-31

How to use this PDF

1. Start with Part 1 if you only want the simple picture.
2. Use Part 2 when you need to open the LMS locally.
3. Use Part 3 when you ask Codex or ChatGPT to change the app.
4. Use Part 4 before touching Git.
5. Use Part 6 before launch.
6. Use the appendices as dictionaries for files, commands, and Git words.

Contents

Note. This contents page is a reading map, not a clickable generated table of contents. The PDF is meant to be opened and skimmed visually.

Part 1 - The Simple Picture

Your LMS is not one single magic file. It is a small system made of a few parts that talk to each other. When something breaks, the goal is to identify which part is failing instead of feeling like the whole app is broken.

Part	Plain-English Meaning	Real Project Location
Frontend	The screens people see and click: website, login, student dashboard, admin pages, quiz pages.	frontend/src/
Backend/API	The server brain. It receives requests, checks users, saves quiz results, talks to the database.	backend/src/
Database	The stored data: users, courses, lessons, questions, quizzes, payments, attempts.	MySQL through XAMPP or production DB
XAMPP/Apache	The local web server that lets the built website open at localhost/lms.	/Applications/XAMPP/
Native app	Android/iOS wrappers around the frontend, using Capacitor.	frontend/android/ and frontend/ios/
Desktop app	Electron wrapper for desktop builds.	desktop/
Git	The project history and safety system.	.git/ and GitHub

The request flow

Most screens follow this flow:

1. User opens a page in the browser or native app.
2. The React frontend displays the screen.
3. If the page needs data, frontend calls the backend API.
4. Backend checks permissions and talks to MySQL.
5. Backend sends data back.
6. Frontend shows the result.

```
Browser or mobile app -> React frontend -> /api backend -> MySQL database -> response back to screen
```

Why local development can still work after adding a real domain

Files such as sitemap.xml, robots.txt, canonical URLs, and social preview tags can point to your real public domain. That does not stop local development. Local development uses localhost and local API settings. SEO metadata is mostly for Google and social previews after deployment.

Type	Example
Local app URL	http://localhost/lms/
Local API URL	http://localhost:3000/api
Production public URL	https://your-real-domain.com/lms or your chosen final URL
Production API URL	https://your-real-domain.com/api

What vibe coding means for ownership

Vibe coding is useful because you can describe what you want and let an AI agent help implement it. The important upgrade is learning enough of the map to ask for safer changes and to know what must be tested.

You do not need to memorize every file. You need to know which area the change belongs to and what result to verify.

> **Owner mindset.** You are allowed to say: 'I do not know the file names. Please inspect the app, find the right files, make the change, run tests, and explain what changed.' That is a good prompt.

Part 2 - How To Open The LMS Locally

Local means the app runs on your own computer instead of the public internet. Your local app is stored inside the XAMPP htdocs folder.

```
/Applications/XAMPP/xamppfiles/htdocs/lms
```

Daily local startup

1. Open XAMPP.
2. Start Apache.
3. Start MySQL.
4. Open Terminal inside the project folder.
5. Start the backend API.
6. Open the login page in the browser.

```
cd /Applications/XAMPP/xamppfiles/htdocs/lms
npm run start:api:bg
npm run status:api
open http://localhost/lms/auth/login
```

The most important local URLs

Area	URL
Public website	http://localhost/lms/
Login	http://localhost/lms/auth/login
Student dashboard	http://localhost/lms/app/dashboard
Admin dashboard	http://localhost/lms/admin/dashboard
API health	http://localhost:3000/api/health
API ready check	http://localhost:3000/api/health/ready

If the login page says Network Error

This usually means the frontend opened, but the backend API is not running or cannot reach the database.

1. Check whether the API is running.
2. Check API health.
3. If needed, restart the API.
4. If health works but ready fails, check XAMPP MySQL and database settings.

```
npm run status:api
curl http://localhost:3000/api/health
npm run stop:api:bg
npm run start:api:bg
```

Why /api alone can show 404

The backend does not have a plain index page at /api. That is normal. Use a real endpoint like /api/health, /api/auth/login, or /api/results.

When frontend changes do not appear

The Apache version uses the built frontend bundle from frontend/dist. After changing frontend files, rebuild the frontend before testing through localhost/lms.

```
npm run build:frontend
```

Simple rule. If you changed what the user sees and localhost/lms still looks old, rebuild the frontend.

Part 3 - Project Folders And What They Mean

You do not need to open every folder. Think of the project like a building. Some rooms are where the app is built. Some are storage. Some are generated and should not be edited by hand.

Folder/File	What It Means	Should You Edit It?
frontend/src/surfaces/website/	Landing page, login, register, legal pages, public AI generator.	Only when changing website/auth/public pages.
frontend/src/surfaces/app/	Student app: dashboard, courses, lessons, quizzes, results, notes, billing.	Only when changing student experience.
frontend/src/surfaces/admin/	Admin dashboard and management pages.	Only when changing admin experience.
frontend/src/shared/	Shared UI, API clients, auth store, layout, platform helpers, styles.	Careful. Changes can affect many pages.
frontend/public/	Public images, icons, robots.txt, sitemap.xml.	Yes for assets and SEO files.
frontend/dist/	Built web output generated by Vite.	No. Rebuild instead.
frontend/dist-capacitor/	Built native output generated for Capacitor.	No. Rebuild/sync instead.
backend/src/modules/	Backend features: auth, courses, questions, quizzes, payments, settings.	Yes, but test carefully.
backend/.env	Local backend secrets and database settings.	Edit locally, never share publicly.
backend/.env.example	Example backend environment template.	Safe template only. No real secrets.
database/	SQL dumps and migrations.	Careful. Database changes need backup.
docs/	Project documentation and guides.	Yes.
scripts/	Build, QA, migration, and helper scripts.	Only when changing workflow.
desktop/	Electron desktop wrapper.	Only for desktop app changes.
frontend/android/ and frontend/ios/	Native app projects generated/managed by Capacitor.	Usually through sync, not manual edits.
node_modules/	Installed packages.	No. Do not edit.
backups/	Saved project/database copies.	Do not casually delete.

Where common changes usually live

Change You Want	Likely Area	Example
Landing page text/design	frontend/src/surfaces/website/pages/	LandingPage.jsx
Login/register UI	frontend/src/surfaces/website/auth/	LoginPage.jsx, RegisterPage.jsx
Student dashboard	frontend/src/surfaces/app/student/dashboard/	StudentDashboardPage.jsx
Quiz taking screen	frontend/src/surfaces/app/student/quizzes/	TakeQuizPage.jsx
Quiz result/review	frontend/src/surfaces/app/student/results/	ResultPage.jsx, ReviewPage.jsx
Admin questions	frontend/src/surfaces/admin/pages/questions/	QuestionsPage.jsx
API request code	frontend/src/shared/api/	auth.api.js, aiNotes.api.js, etc.
Dark mode colors	frontend/src/shared/styles/	CSS files and variables
Backend quiz logic	backend/src/modules/quiz-attempts/	controller/service files
Payment settings	backend/src/modules/subscriptions/ and settings	PayHere flows
Email/password reset	backend/src/modules/auth/ and settings	SMTP settings
API IP/domain config	frontend env files and shared platform config	.env.capacitor, config files

Files recently important in your project

- frontend/src/surfaces/app/student/quizzes/TakeQuizPage.jsx - quiz-taking and finish animation behavior.
- frontend/src/shared/seo/PageMeta.jsx - page title, description, canonical, social preview metadata.
- frontend/public/robots.txt and frontend/public/sitemap.xml - Google crawl guidance.
- frontend/src/surfaces/website/pages/RefundPolicyPage.jsx and CookiePolicyPage.jsx - legal pages.
- frontend/src/shared/platform/config.js - platform/API fallback configuration.
- scripts/build/sync-root-index.mjs - copies built frontend files to the root served by Apache.

Part 4 - Git, Main Branch, And Sub-Branches

Git is the save-history system for your app. GitHub is the online place where that history can be stored. A branch is like a separate working lane. You can try changes on a branch without immediately changing the stable main lane.

> **Important correction.** A branch is not really a folder inside main. It is more like a named timeline. People may say 'sub-branch', but Git usually just says branch.

Git Word	Plain-English Meaning
Repository	The full project with history.
main branch	Usually the stable production branch. Treat it as the clean copy.
feature branch	A separate branch for one fix or feature.
current branch	codex/backup-global-card-style-20260530-191847
commit	A saved checkpoint with a message.
push	Upload your commits to GitHub.
pull	Download latest commits from GitHub.
merge	Bring one branch's changes into another branch.
pull request	A GitHub review page before merging.
remote	https://github.com/laheesahamed-cloud/lms.git

Simple branch example

1. main has the current stable app.
2. Create a branch called fix-login.
3. Change login code on fix-login.
4. Run tests.
5. Commit the change.
6. Push branch to GitHub.
7. Merge fix-login into main only when you are happy.

Commands you can safely ask Codex to run

Goal	Command
Check current branch	git branch --show-current
Check changed files	git status --short
See a summary of edits	git diff --stat
Create a new branch	git switch -c codex/short-change-name
Switch branch	git switch branch-name
Save changes	git add selected-files then git commit -m "message"
Upload branch	git push -u origin branch-name

Git safety rules

- Do not commit real secrets such as database passwords, API keys, PayHere secrets, SMTP passwords, or private keys.
- Do not run destructive commands unless you fully understand them.
- Avoid git reset --hard unless you intentionally want to throw away local work.
- Before major changes, ask Codex to check git status and explain what files are already modified.
- Use one branch per meaningful change when possible.

- Commit only after build/tests pass, unless the commit is intentionally a work-in-progress.

> **Best prompt.** Before editing, say: 'Check git status first. Do not overwrite unrelated changes. Make a safe branch if needed. Then fix the issue and run tests.'

Part 5 - How Changes Should Be Made Safely

Every app change should follow a simple loop: understand, edit, build, test, explain. This protects you from accidental breakage, especially when using AI coding tools.

The safe change loop

1. Describe the change in plain English.
2. Ask Codex to inspect the relevant files before editing.
3. Ask Codex to make the smallest reasonable change.
4. Run frontend build if the user interface changed.
5. Run backend build or tests if API/database/auth/payment changed.
6. Open the app and manually test the exact user flow.
7. Ask for a summary of changed files and remaining risks.

Which checks to run

Type Of Change	Minimum Check	Why
Text, color, layout	npm run build:frontend	Confirms React/CSS build still works.
Login/register/auth	npm test and manual login/register	Auth bugs can lock users out.
Quiz submit/results	npm test plus manual student quiz flow	Scoring/results must be trusted.
Admin content	npm run build:backend and manual admin CRUD	Admin data changes affect courses/questions.
Payment	Backend tests plus sandbox/live payment test	Money flow must be tested outside code only.
Email reset	SMTP test with real email	Password reset depends on external email provider.
Native app	npm run mobile:cap:sync then device test	Native bundle can lag behind web code.
Production deploy	npm test, npm run build, health checks, manual role checks	Final safety gate.

What changed recently in plain English

- Quiz finish animation was made more expressive and centered, with review transition behavior.
- Native app support was synced after frontend changes.
- Local native API IP was adjusted for your network, including 172.20.10.2 during testing.
- Dark mode cards were made darker while keeping page backgrounds mostly unchanged.
- SEO metadata, robots.txt, sitemap.xml, refund policy, and cookie policy were added.

- A fake Google sign-in button was removed so users do not click an unfinished login option.

Manual test checklist after a UI change

- Open public website.
- Open login page.
- Login as student.
- Open dashboard.
- Start a quiz.
- Finish quiz and check finish animation.
- Confirm automatic review/result navigation works.
- Switch light/dark mode if available.
- Check on narrow/mobile width.
- Logout and login as admin.
- Check admin dashboard and one content page.

Part 6 - Launch Readiness

Launch is not only code. A real launch needs domain, HTTPS, production environment variables, database backups, payment setup, email setup, legal pages, monitoring, and manual testing.

Critical blockers before launch

Blocker	What Must Be True	Who/Where
Domain	A real domain is connected and replaces your-domain.com placeholders.	Hosting/DNS and project env files.
HTTPS	Website and API load over https, not plain http.	Hosting/reverse proxy.
Production API	Frontend calls the public production API, not localhost or LAN IP.	Frontend env/build.
Database	Production DB exists with correct user, password, schema, and backups.	Hosting/MySQL.
Secrets	JWT/session/settings keys are long and private.	Backend environment.
Email	Password reset sends real email successfully.	SMTP settings.
Payment	PayHere live payment and notify webhook are tested end to end.	Admin settings/PayHere.
Legal	Privacy, Terms, Refund, Cookie pages contain real business information.	Website pages and legal review.
Monitoring	Uptime/error alerts exist for API, database, payment, email, and disk space.	Hosting/monitoring tool.
Manual QA	Student, admin, logged-out, mobile, and payment flows are tested.	You/Codex/testing person.

Files with domain placeholders

- frontend/index.html

- frontend/public/robots.txt
- frontend/public/sitemap.xml
- backend/.env.example and production backend env
- frontend/.env.example and production frontend env
- frontend/.env.capacitor.example and native production env

Production env values in plain English

Variable	Meaning
FRONTEND_URL	Your public website URL.
FRONTEND_URLS	Allowed browser origins that may call the API.
APP_PUBLIC_URL	Public LMS app URL used in links.
API_PUBLIC_URL	Public backend API URL.
ALLOW_LAN_ORIGINS	Should be false in production.
SETTINGS_ENCRYPTION_KEY	Long secret used to protect saved settings.
HEALTH_METRICS_TOKEN	Secret token for protected metrics.
DB_HOST/DB_USER/DB_PASSWORD/DB_NAME	Production database connection details.
VITE_API_BASE_URL	Frontend's API target.
VITE_PUBLIC_WEBSITE_URL	Public website URL used by metadata.

What not to launch with

- Do not launch with localhost or LAN IP in production frontend API settings.
- Do not launch with ALLOW_LAN_ORIGINS=true.
- Do not launch with DB_USER=root or empty database password.
- Do not launch if password reset email is untested.
- Do not launch if PayHere live payment and notify webhook are untested.
- Do not launch without a recent database backup.
- Do not launch with example legal policy text only.

> **Launch truth.** A build passing means the code can compile. It does not prove payments, email, DNS, SSL, backups, or legal details are ready.

Part 7 - Native App, API IPs, And Local Testing

The native app is built with Capacitor. Capacitor packages the React frontend into Android/iOS projects. The native app still needs to call the backend API, so API URLs matter a lot.

Why localhost can fail on a phone

On your Mac, localhost means your Mac. On a phone, localhost means the phone itself. So a phone usually cannot call http://localhost:3000/api unless the backend is running on the phone, which it is not.

Situation	API URL
Browser on Mac	http://localhost:3000/api can work.
Phone on same Wi-Fi	Use your Mac's LAN IP, such as http://172.20.10.2:3000/api when that is the current IP.
Production native app	Use https://your-real-domain.com/api.

Native sync command

After frontend/native changes, rebuild and sync Capacitor.

```
npm run mobile:cap:sync
npm run mobile:cap:sync:android
npm run mobile:cap:sync:ios
```

If the native app shows old UI

1. Rebuild/sync Capacitor.
2. Open Android Studio or Xcode again.
3. Clean/reinstall app if needed.
4. Confirm the device can reach the backend API URL.

> **Use HTTPS in production.** A shipped native app should call the public HTTPS API. A local IP is only for testing on your own network.

Part 8 - Troubleshooting

Use this when something breaks. Start with the simplest checks before assuming the app is ruined.

Symptom	Most Likely Cause	First Fix
Login says Network Error	Backend API is stopped or unreachable.	npm run start:api:bg then curl /api/health.
API health works but ready fails	Database/MySQL problem.	Start XAMPP MySQL and check backend .env.
/api shows 404	No plain API index route.	Use /api/health instead.
Frontend change not visible	Built bundle is old.	npm run build:frontend.
Native app old UI	Capacitor not synced or app not reinstalled.	npm run mobile:cap:sync.
Phone cannot reach API	Using localhost instead of computer IP.	Use current LAN IP or production HTTPS API.
Admin gets 403	Role/permission problem.	Check user role and backend permission mapping.
Results not showing	Quiz attempts/student answers issue.	Check quiz-attempts and results API.
Payment not activating	PayHere notify/webhook not reaching backend.	Check public notify URL and logs.
Password reset email not arriving	SMTP setting or provider issue.	Test SMTP settings and logs.

Log locations

- .runtime/api.log - background API log.
- Browser DevTools Network tab - failing frontend requests.
- Backend terminal output - API errors during development.
- Hosting provider logs - production backend errors.

First three commands when confused

```
npm run status:api
curl http://localhost:3000/api/health
npm run build:frontend
```

If those are clean, then the problem is more specific and should be traced by page/API route.

Appendix A - File And Folder Dictionary

This section is intentionally simple. Use it when someone mentions a file name and you want to know what it is.

Name	Meaning	Beginner Warning
package.json	A command/menu file for Node projects. It lists scripts like build and test.	Edit only when changing commands or dependencies.
node_modules	Installed packages downloaded by npm.	Never edit manually.
src	Source code written by developers/AI.	This is where real code lives.
dist	Built output generated from source.	Do not edit; rebuild.
.env	Private local settings and secrets.	Do not upload/share real secrets.
.env.example	A template showing what env variables exist.	Safe only if it has no real secrets.
README.md	Human-readable project notes.	Good first file to read.
router.jsx	Frontend route map: which URL opens which page.	Wrong routes can break navigation.
AppFrame/AppShell	Main app layout wrapper.	Changes affect many pages.
BootLoader	Loading screen when app starts.	Affects perceived startup polish.
controller.ts	Backend file that receives API requests.	Usually paired with service.ts.
service.ts	Backend file where business logic often lives.	Changes can affect data behavior.
schema-sync.service.ts	Database schema bootstrap/backfill logic.	Be careful; database structure risk.
capacitor.config.ts	Native app config.	Important for mobile builds.
robots.txt	Tells search engines what they may crawl.	Use real domain at launch.
sitemap.xml	List of important public URLs for search engines.	Use real domain at launch.

Appendix B - Commands Cheat Sheet

Command	Use It For	When
npm run install:all	Install frontend/backend dependencies.	First setup or after package changes.
npm run start:api	Start API in current terminal.	Development when terminal stays open.
npm run start:api:bg	Start API in background.	Daily local use.
npm run status:api	Check background API status.	When login/API fails.
npm run stop:api:bg	Stop background API.	Restarting API.
npm run dev:frontend	Start Vite dev server.	Optional frontend development.
npm run build:frontend	Build frontend.	After UI changes.
npm run build:backend	Build backend.	After API/backend changes.
npm run build	Build frontend and backend.	Before committing/deploying.
npm test	Run project QA/security checks.	Before commit/launch.
npm run mobile:cap:sync	Build and sync native bundle.	After native/frontend changes.
npm run check:health	Check local API health.	When diagnosing API.
git status --short	See changed files.	Before and after edits.
git branch --show-current	See branch name.	Before committing.

Appendix C - Environment Variables In Plain English

Environment variables are settings outside the code. They tell the app where the database is, where the API is, and which secret keys to use. Production environment variables are usually set in the server or hosting dashboard, not typed into public GitHub files.

Backend variables

Backend Variable Group	Meaning
NODE_ENV	Use production on the live server.
PORT	Backend server port, usually 3000 locally.
FRONTEND_URL/FRONTEND_URLS	Allowed frontend domains that can call the API.
APP_PUBLIC_URL	Public URL used in generated links.
API_PUBLIC_URL	Public API URL.
BODY_LIMIT	Max request body size.
ALLOW_LAN_ORIGINS	Allows local network origins. False in production.
SETTINGS_ENCRYPTION_KEY	Protects saved provider/payment/email settings.
HEALTH_METRICS_TOKEN	Protects metrics endpoint.
OPENROUTER/GEMINI keys	AI provider credentials.
VAPID/APNS/FCM	Push notification credentials.
DB_*	MySQL database connection.

Frontend variables

Frontend Variable	Meaning
VITE_API_BASE_URL	Main API URL the frontend calls.
VITE_API_BASE_URLS	Fallback API URLs.
VITE_API_REQUEST_TIMEOUT_MS	How long frontend waits for API response.
VITE_PWA_SW_URL/SCOPE	Progressive Web App service worker paths.
VITE_LMS_BUILD_TARGET	web, pwa, native, or desktop build target.
VITE_ENABLE_PWA	Whether PWA features are enabled.
VITE_PUBLIC_WEBSITE_URL	Public website URL used by metadata.
VITE_APP_ONLY_HOSTS	Domains that should open app-only surface.

Secret rule. If a value can give access to money, user data, database, email, AI billing, or private admin functions, treat it as a secret.

Appendix D - What To Ask Codex

Good prompts tell Codex the result you want, the risk level, and what checks to run.

General safe fix prompt

```
I do not know the file names. Please inspect the project, find the right files, fix this issue, do not overwrite unrelated changes, then run the relevant build/tests and explain what changed.
```

UI change prompt

```
Make this UI change: [describe]. Keep light/dark mode working. Check desktop and mobile layout. Run npm run build:frontend. Tell me which files changed.
```

Backend/API prompt

```
Fix this API/backend issue: [describe]. Check the route/controller/service. Run backend build and relevant tests. Explain any database risk.
```

Launch readiness prompt

```
Review launch readiness again. Check domain placeholders, env files, SEO files, legal pages, payment/email readiness, security, tests, and tell me what is still blocked.
```

Git prompt

```
Before editing, check git status and current branch. If needed, create a branch with codex/ prefix. After the fix, show changed files and ask before committing.
```

Appendix E - 30 Day Learning Path For Owning This App

This is not a computer science course. It is a practical path so you can talk to developers and AI tools with confidence.

Days	Focus	Goal
1-3	Open app locally	Start XAMPP, API, login page, health check without panic.
4-6	Folder map	Know frontend, backend, database, docs, scripts, native folders.
7-9	Git basics	Understand main, branch, commit, push, pull, merge.
10-12	Frontend basics	Know where pages, shared components, styles, and API clients live.
13-15	Backend basics	Know controller, service, module, env, database connection.
16-18	Debug basics	Use API health, build output, logs, browser Network tab.
19-21	Launch basics	Understand domain, HTTPS, env vars, backups, monitoring.
22-24	Payments/email	Understand why PayHere and SMTP require real external testing.
25-27	Native basics	Understand localhost vs LAN IP vs production HTTPS API.
28-30	Safe AI workflow	Ask better prompts, demand tests, read summaries, avoid secrets.

The only things to memorize first

- Project folder: /Applications/XAMPP/xamppfiles/htdocs/lms
- Local login: http://localhost/lms/auth/login
- API health: http://localhost:3000/api/health
- Build frontend: npm run build:frontend
- Run QA: npm test
- Check branch: git branch --show-current
- Check changed files: git status --short

Appendix F - Important Routes In The App

Routes are URL paths. A route tells the frontend which page to open. The same built React app serves the public website, student app, and admin console.

Area	Route	What It Opens
Website	/	Public landing website.
Website	/login or /auth/login	Login page.
Website	/register or /auth/register	Register page.
Website	/terms	Terms page.
Website	/privacy-policy	Privacy policy.
Website	/refund-policy	Refund policy.
Website	/cookie-policy	Cookie policy.
Website	/ai	AI question builder page.
Student	/app/dashboard	Student dashboard.
Student	/app/courses	Course list.
Student	/app/quizzes	Practice quizzes.
Student	/app/results	Student results list.
Student	/app/review/:attemptId	Quiz review page.
Admin	/admin/dashboard	Admin dashboard.
Admin	/admin/questions	Question bank.
Admin	/admin/quizzes	Quiz management.
Admin	/admin/users	User management.
Admin	/admin/settings	Settings.

Important API endpoints

Method	API Route	What It Is For
GET	/api/health	Basic API health check.
GET	/api/health/ready	API plus database readiness check.
POST	/api/auth/login	Login request.
POST	/api/auth/register	Register request.
GET	/api/auth/me	Current logged-in user.
POST	/api/auth/forgot-password	Start password reset.
GET	/api/courses/student	Student course list.
GET	/api/quiz-attempts/quizzes	Student quiz list.
POST	/api/quiz-attempts/exam/:quizId/submit	Submit exam quiz.
GET	/api/results/:attemptId	Result detail.
POST	/api/subscriptions/payhere/initiate	Start PayHere payment.
POST	/api/subscriptions/payhere/notify	PayHere webhook/notification.

Appendix G - Manual QA Script Before Launch

Manual QA means a human actually opens the app and behaves like a real user. Automated tests help, but they cannot prove every payment, email, mobile, and design experience.

Logged-out visitor

- Open public website.
- Check top navigation links.
- Open login and register pages.
- Open Terms, Privacy, Refund, and Cookie pages.
- Check mobile width.
- Confirm no unfinished or placeholder copy appears.

Student user

- Login as student.
- Open dashboard.
- Open courses.
- Open a lesson.
- Open quizzes.
- Take a quiz from start to finish.
- Watch finish animation.
- Confirm result/review page opens correctly.
- Check billing/subscription page.

- Logout.

Admin user

- Login as admin.
- Open admin dashboard.
- Open courses, structure, users, questions, quizzes, subscriptions, reports, settings.
- Create/edit one safe test item if using a staging database.
- Confirm unauthorized users cannot open admin pages.
- Logout.

Production services

- Send a password reset email through real SMTP.
- Complete one low-value live PayHere payment.
- Confirm PayHere notify webhook activates the subscription.
- Confirm database backup exists before launch.
- Confirm uptime monitoring is enabled.
- Confirm error logs are visible.

Appendix H - Beginner Glossary

Word	Meaning	Why It Matters
API	A URL-based way for frontend and backend to talk.	Most data problems happen here.
Backend	Server code that handles logic and data.	Protects users, quizzes, payments, settings.
Build	Turns source code into runnable output.	Needed after code changes.
Cache	Stored old files/data.	Can make old UI appear.
CORS	Browser rule about which websites can call API.	Wrong CORS can block login/API.
Database	Stored app data.	Needs backups and careful changes.
Deploy	Put app onto live server.	Launch step.
DNS	Domain name routing.	Needed for real domain.
Environment variable	Private setting outside code.	Keeps secrets/config out of source.
Frontend	Visible user interface.	Most design changes live here.
HTTPS	Secure website connection.	Required for production trust.
localhost	This same machine.	Works on Mac, not automatically on phone.
Migration	Database structure update.	Needs backup before production.
Native app	Android/iOS app wrapper.	Needs correct API URL.
Node/npm	JavaScript runtime and package manager.	Runs build/test commands.
Pull request	Review page before merging branch.	Safer Git workflow.
Reverse proxy	Server rule forwarding /api to backend.	Needed in production.
SEO	Search engine basics.	Needs titles, sitemap, robots.
SMTP	Email sending service.	Needed for password reset.
Webhook	External service calls your backend.	PayHere uses notify webhook.

Appendix I - Handover Sheet For A Hosting Person

If you hire someone or ask a hosting support person to deploy the app, give them this list. It prevents vague conversations.

- Project type: React frontend plus NestJS backend plus MySQL database.
- Local project path: /Applications/XAMPP/xamppfiles/htdocs/lms.
- Frontend build command: npm run build:frontend.
- Backend build command: npm run build:backend.
- Full build command: npm run build.
- QA command: npm test.
- Backend port: 3000 by default.
- API base path: /api.

- Frontend build output: frontend/dist.
- Backend build output: backend/dist.
- Production must reverse proxy /api to the backend.
- Production must serve frontend through HTTPS.
- Production database should not use root user.
- Backups must include MySQL database and upload folders.
- PayHere notify URL must be public HTTPS and point to backend.
- Native app production API must be public HTTPS, not local IP.

Final reminder. You can own this app by using maps, checklists, backups, and safe Git branches. You do not need to memorize every file name. You need to ask for careful changes and verify the important flows.